

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 6

Выполнила:

Красюк Карина

Группа К3441

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Задача.....	3
Ход работы	3
Вывод	6

Задача

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Вариант “Сайт для поиска работы”.

Ход работы

Листинг 1 – Подключение Bull в app.module.ts

```
@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
    }),
    BullModule.forRoot({
      redis: {
        host: 'localhost',
        port: 6379,
      },
    }),
    UsersModule,
  ],
})
```

Модуль импортирует:

- ConfigModule для глобальной конфигурации;
- BullModule для работы с очередями на Redis;
- UsersModule — пользовательский модуль приложения.

Листинг 2 – Создание очереди в process.ts

```
import { Process, Processor } from '@nestjs/bull';
import bull from 'bull';

@Processor('quiz')
export class QuizProcessor {
  @Process('processQuiz')
```

```

    async handleProcessQuiz(job: bull.Job<{ quizId: string
}>): Promise<{ status: string; quizId: string }> {
        console.log('Обработка квиза:', job.data.quizId);

        await new Promise(resolve => setTimeout(resolve,
3000));

        return { status: 'completed', quizId: job.data.quizId
};
    }
}

```

Процессор для очереди Bull обрабатывает задания для квизов, который:

- Обрабатывает задания из очереди quiz;
- Метод handleProcessQuiz выполняет задание типа processQuiz;
- Имитирует обработку с задержкой 3 секунды;
- Возвращает объект со статусом завершения и ID квиза.

Листинг 3 – Регистрация очереди в модуле quiz.module.ts

```

@Module({
  imports: [
    BullModule.registerQueue({
      name: 'quiz',
    }),
  ],
})

```

Фрагмент модуля регистрирует очередь Bull с именем quiz.

Листинг 4 – Добавление логики в сервис quiz.service.ts

```

import { Injectable } from '@nestjs/common';
import { InjectQueue } from '@nestjs/bull';
import * as bull from 'bull';

@Injectable()
export class QuizService {

```

```

    constructor(@InjectQueue('quiz') private quizQueue:
bull.Queue) {}

    async addQuizToQueue(quizId: string): Promise<void> {
        await this.quizQueue.add('processQuiz', { quizId }, {
            attempts: 3,
            backoff: 1000,
        });
    }
}

```

Сервис добавляет задания в очередь Bull и:

- Внедряет очередь quiz через декоратор `@InjectQueue`;
- Метод `addQuizToQueue` добавляет задание типа `processQuiz` с параметрами:

1. `quizId` - идентификатор квиза
2. `attempts: 3` - максимальное количество попыток выполнения
3. `backoff: 1000` - задержка между повторными попытками (1 секунда)

Листинг 5 – Добавление логики в контроллер `quiz.controller.ts`

```

@Post('/:id/process')
async processQuiz(@Param('id') id: string) {
    await this.quizService.addQuizToQueue(id);
    return { message: 'Задача добавлена в очередь' };
}

```

Endpoint контроллера с декоратором `@Post`, который:

- Принимает параметр `id` из URL;
- Вызывает сервис для добавления квиза в очередь на обработку;
- Возвращает сообщение о том, что задача была добавлена в очередь.

Вывод

В данной лабораторной работе я научилась создавать и работать с очередью Bull.